



Intel® Agilex™ FPGA

10nm
FPGA Fabric

EMIB

PCIe Gen 5

Output
chipset

Diseño Digital Avanzado

Unidad 6 - Memoria BRAM y FSM

Dr. Ariel L. Pola
ariel.pola@mi.unc.edu.ar
December 14, 2024

Tabla de Contenidos

1. Proyecto FSM y BRAM

2. Plataforma de Verificación



The image shows a top-down view of an Intel Agilex FPGA chip. The chip is a square package with various components labeled on its surface. At the top, it says 'Intel® Agilex™ FPGA'. Below this, there's a large black square labeled '10nm FPGA Fabric'. To the left of the fabric is a blue chip labeled 'CPU Cores' and a green chip labeled 'HBM'. To the right is a red chip labeled 'Other Chiplet'. Below the fabric is a silver chip labeled 'DDR'. There are also several yellow chips labeled 'EMIB' (Embedded Multi-Die Interconnect Bridge) connecting different parts of the chip. A green chip labeled 'PCIe Gen 5' is at the bottom right. The background is a dark blue network of lines and nodes.

Proyecto FSM y BRAM

Proyecto FSM y BRAM

- El proyecto consiste en implementar una memoria BRAM que almacene los datos provenientes de un contador, el cual puede cambiar el paso de incremento.
- Los datos se guardan a la frecuencia de clock.
- BRAM
 - La memoria comienza a loguear cuando detecta el flanco ascendente de "log_in_ram_run_from_micro" y guarda datos en cada ciclo de clock hasta llegar a la posición final de la memoria.
 - Cuando esto ocurre se pone en alto "log_out_full_from_ram" para avisar al usuario que puede comenzar a leer los datos almacenados.
 - La BRAM es una memoria de doble puerto, los cuales usan el mismo clock de referencia.
 - Al puerto de entrada de datos (interno al módulo) se conecta un contador con paso configurable (se detalla más abajo). La dirección de memoria donde se almacena este dato es controlado por otro contador, el cual se detiene cuando se llega al final de la memoria.
 - Para leer los datos desde el micro se coloca primero la dirección a leer "log_read_addr_from_micro" y esto retorna el dato en el puerto de salida "log_data_from_ram".

Proyecto FSM y BRAM

■ FSM

- El control de la BRAM se implementa con una FSM de Moore con los estados WAIT_INIT y WRITTING.
- En caso de ser necesario agregar mas estados.

■ Contador con paso configurable

- Dentro del modulo diseñado se instanciará un contador que cambie el paso de incremento según los bits de "rf_enables_module"

rf_enables_module[0]: $c = c + 1$

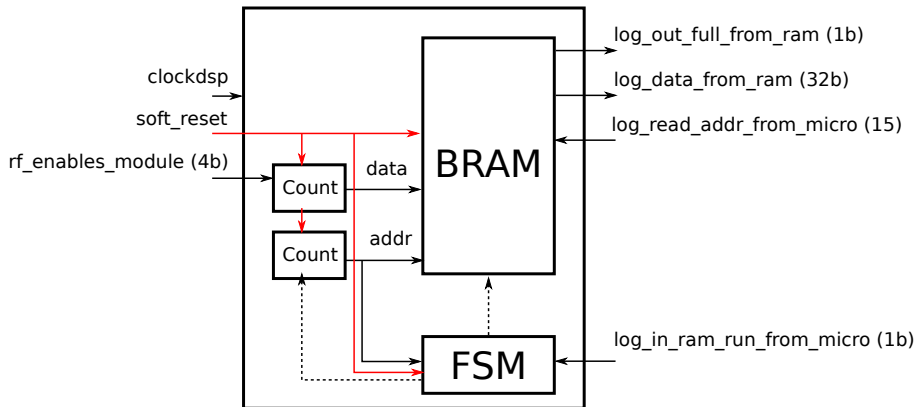
rf_enables_module[1]: $c = c + 2$

rf_enables_module[2]: $c = c + 3$

rf_enables_module[3]: $c = c + 4$

■ La señal de reset del módulo diseñado se controla con "soft_reset" (1).

Proyecto FSM y BRAM



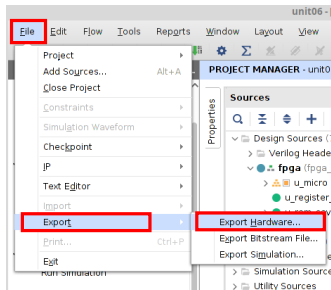
Plataforma de Verificación



Plataforma de Verificación

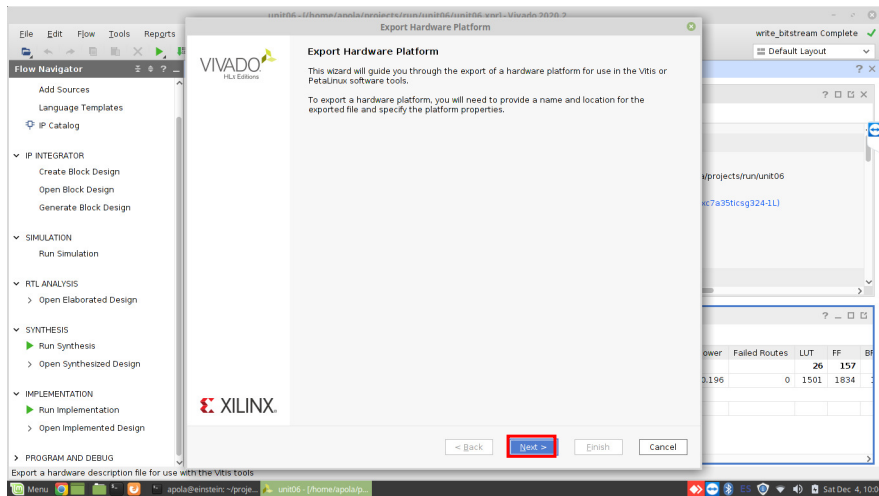
- A continuación se detallan los pasos para generar el binario y compilar el software que se ejecuta en el microprocesador instanciado.
- 1 Incluir el archivo **fpga_p6.v**, **design_1.bd** y **Arty_Master.xdc**.
- 2 Modificar los path de los comandos **include** apuntando a los archivos locales.
- 3 Generar el **bitstream**.
- 4 Exportar el **hardware**. **File-Export-Export Hardware**. Hacer click en el box de incluir el bitstream.
- Ejecutar la herramienta Vitis desde el Vivado siguiendo los pasos que se detallan a continuación.

Crear la aplicación en Vitis



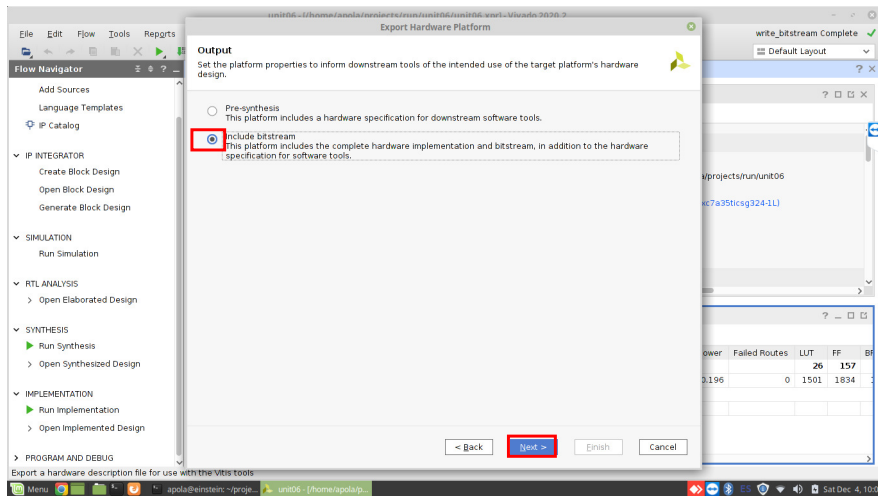
Exportar el hardware desde la opción “File” de la barra de herramientas.

Crear la aplicación en Vitis



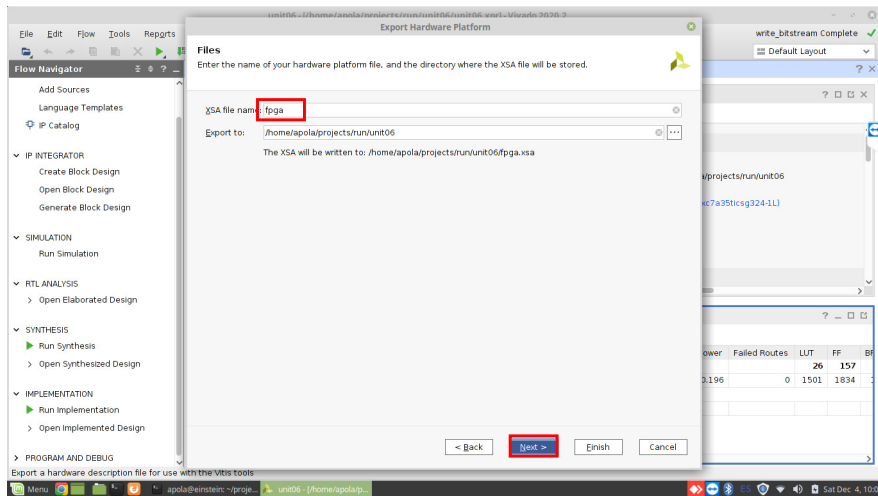
Pulsar NEXT.

Crear la aplicación en Vitis



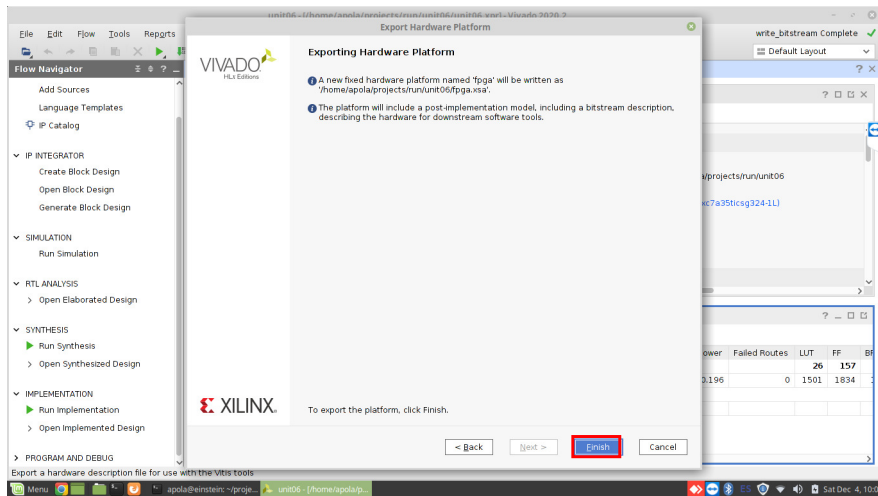
Incluir el bitstream.

Crear la aplicación en Vitis



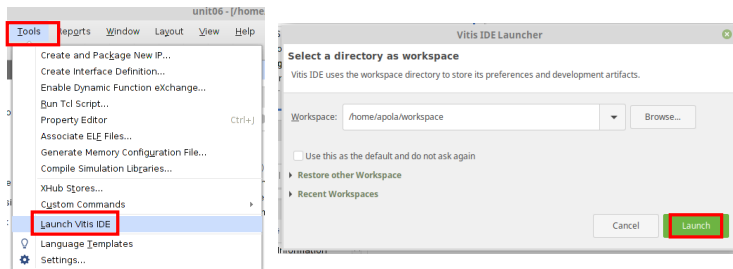
Definir el nombre del XSA (dejarlo por defecto).

Crear la aplicación en Vitis



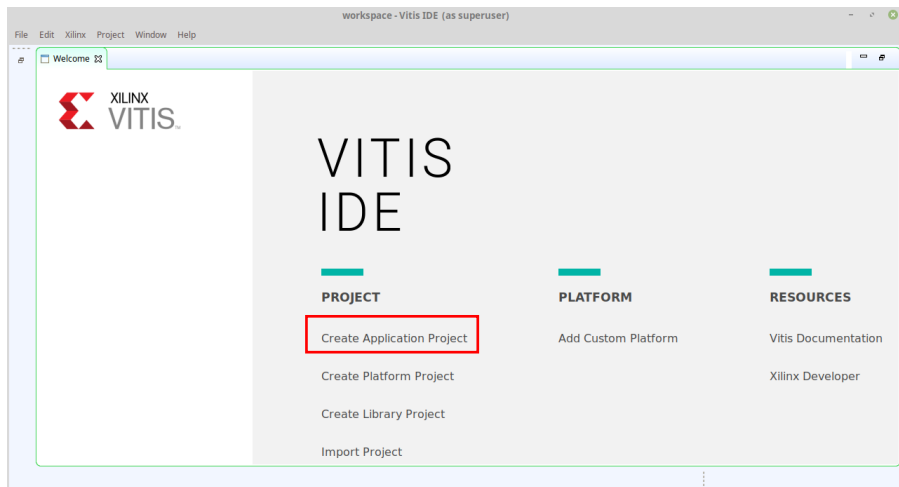
Pulsar FINISH.

Crear la aplicación en Vitis



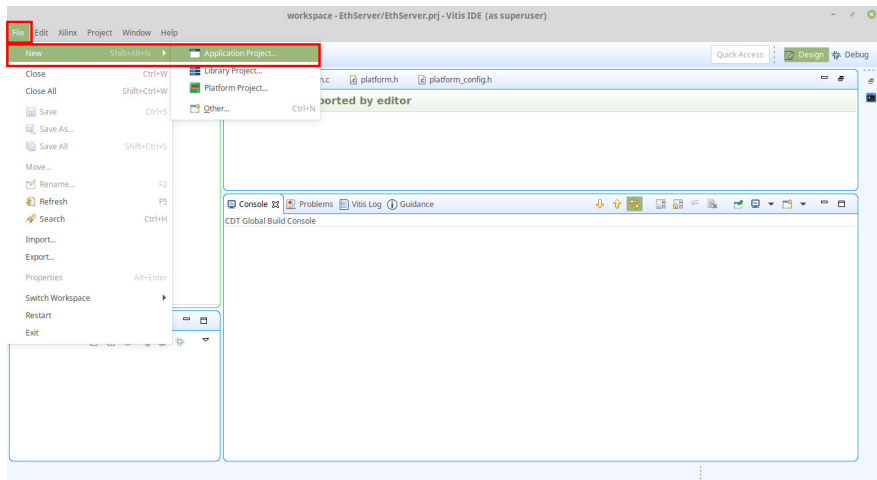
Ejecutar Vitis desde Vivado, seleccionando la opción Launch Vitis IDE.

Crear la aplicación en Vitis



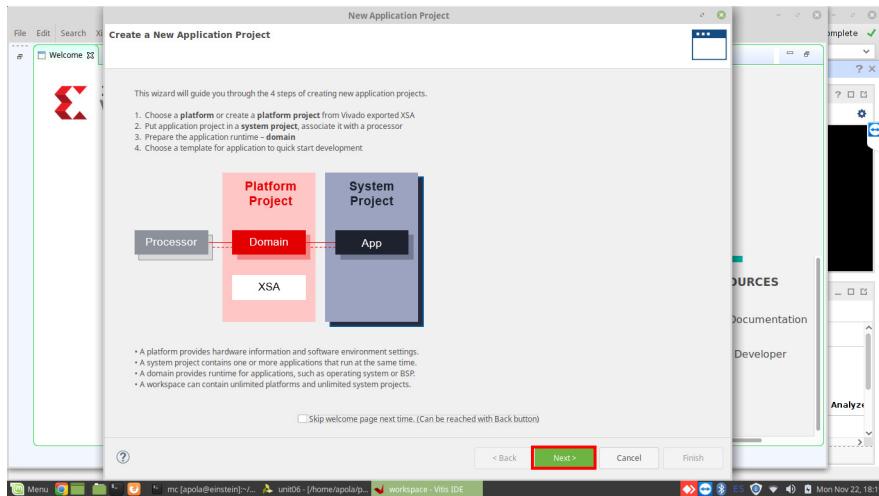
Entorno de trabajo en Vitis. Opción de inicio 1: En la pagina de inicio seleccionar Create Application Project.

Crear la aplicación en Vitis



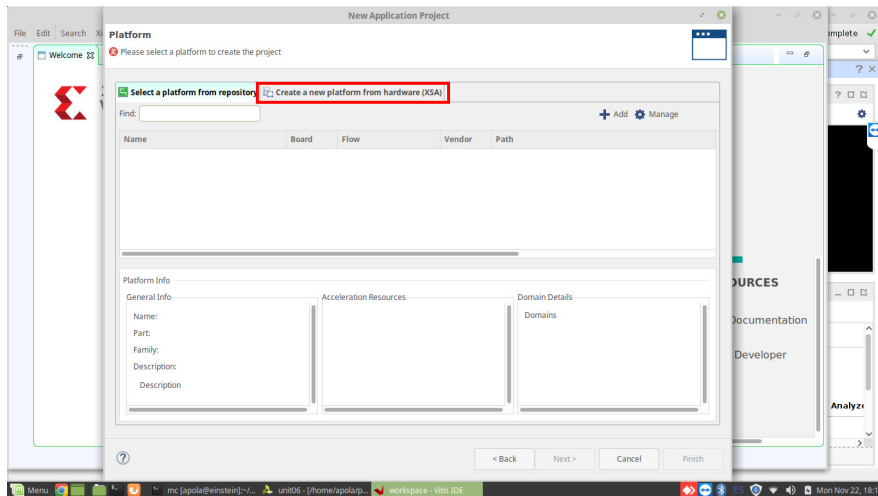
Entorno de trabajo en Vitis. Opción de inicio II: También puede iniciarse la nueva aplicación desde File-New-Application Project.

Crear la aplicación en Vitis



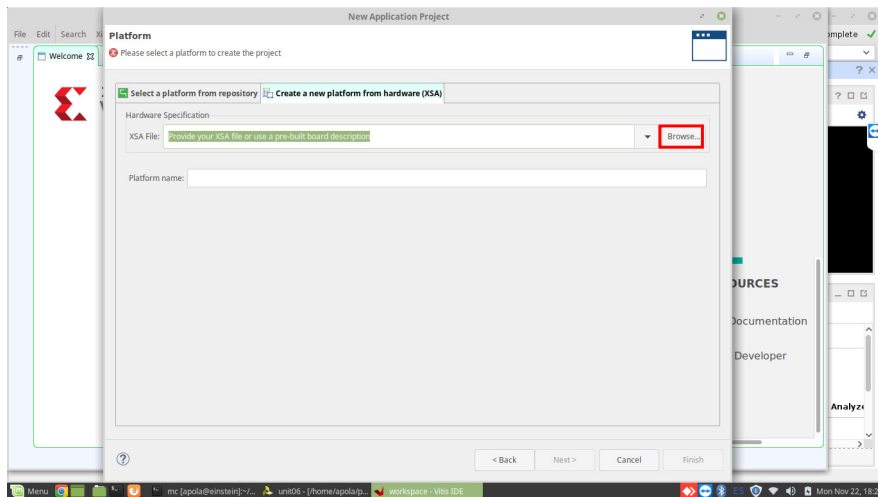
Crear un nuevo proyecto.

Crear la aplicación en Vitis



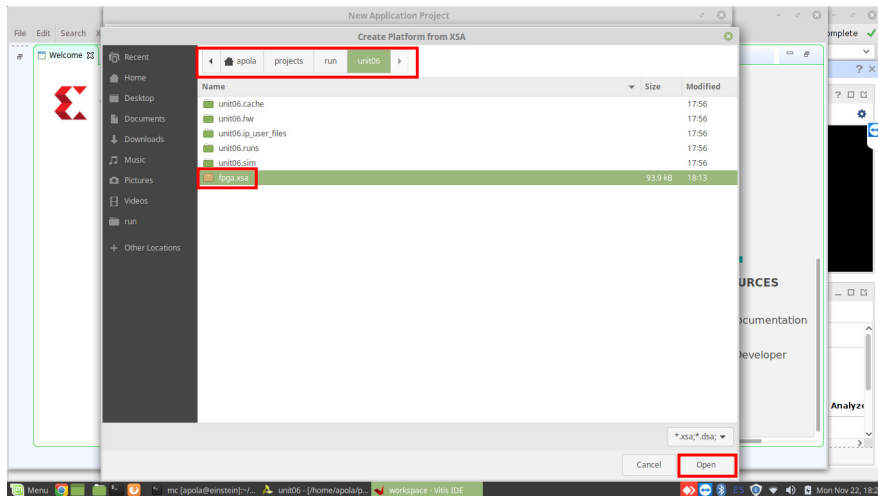
Crear una nueva plataforma importando el archivo XSA generado con el Vivado.

Crear la aplicación en Vitis



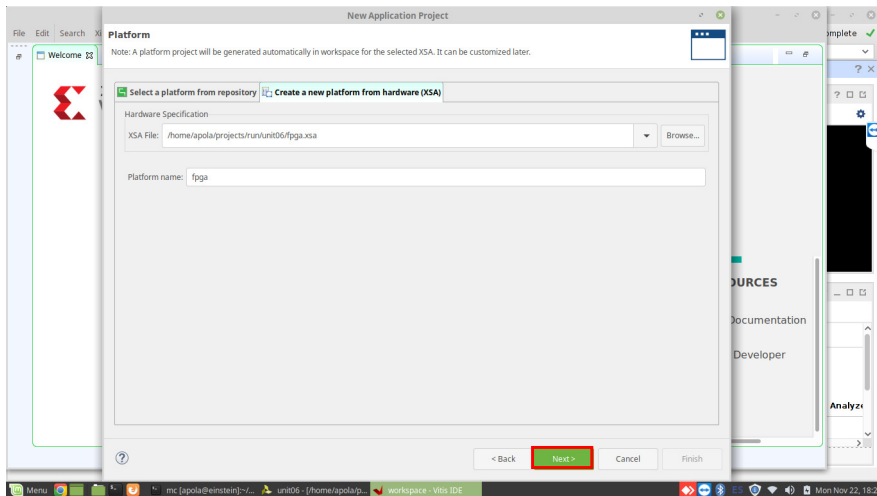
Crear una nueva plataforma importando el archivo XSA generado con el Vivado.

Crear la aplicación en Vitis



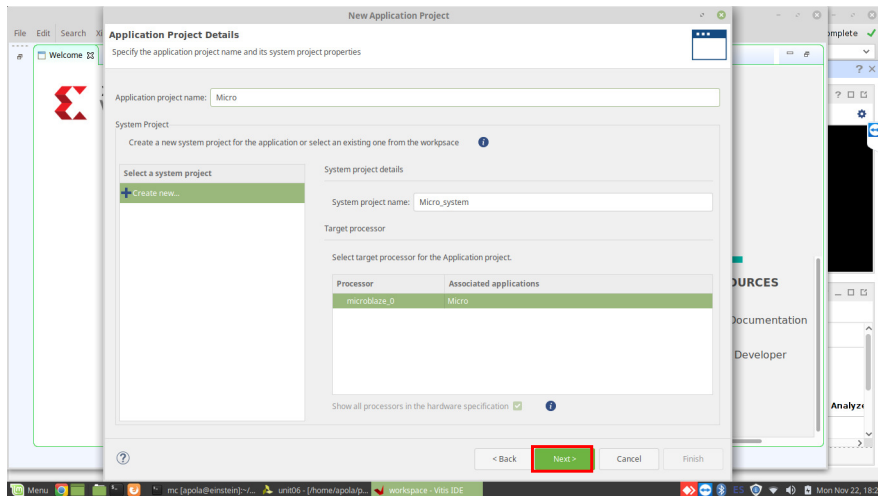
Crear una nueva plataforma importando el archivo XSA generado con el Vivado.

Crear la aplicación en Vitis



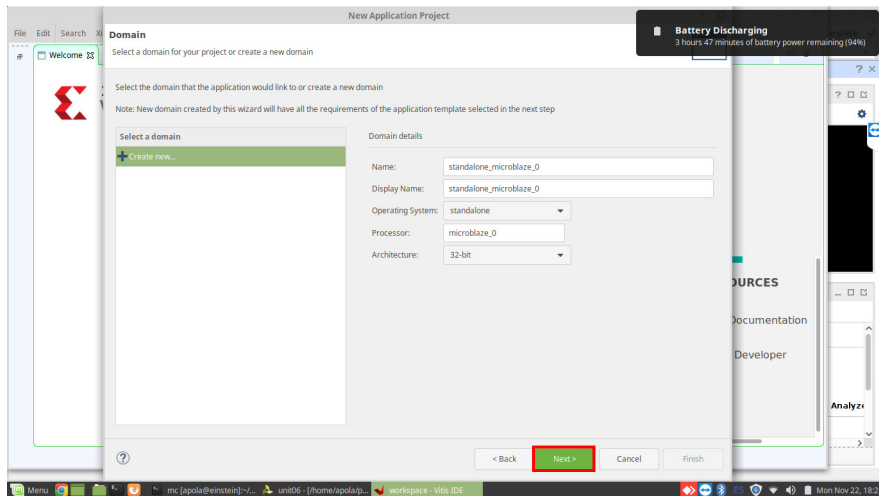
Crear una nueva plataforma importando el archivo XSA generado con el Vivado.

Crear la aplicación en Vitis



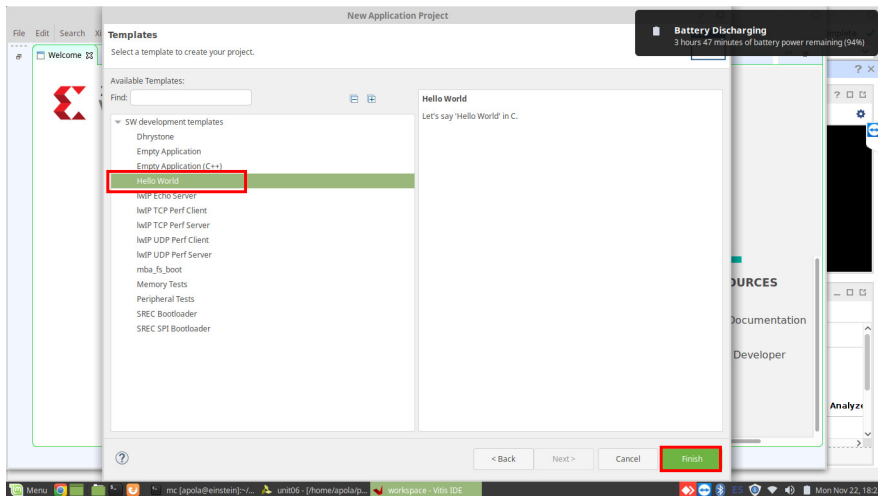
Asignar el nombre de la Aplicación (Ej. Micro).

Crear la aplicación en Vitis



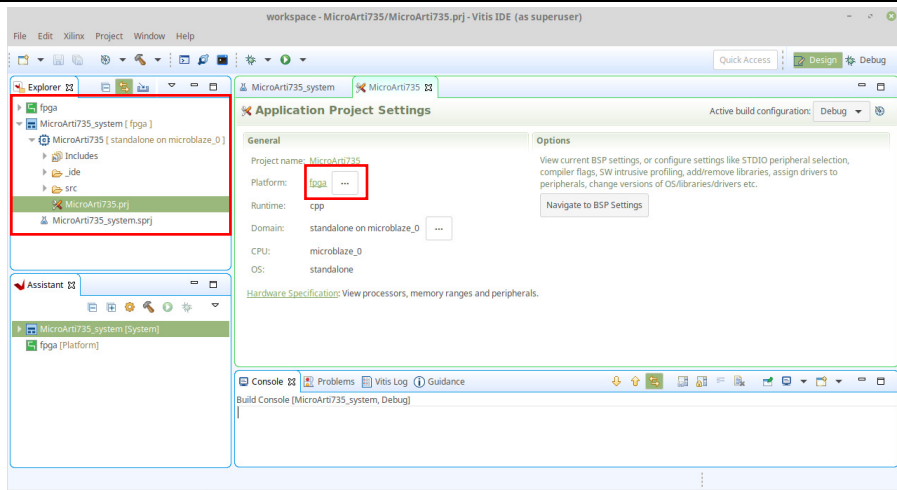
Verificar las características del Micro.

Crear la aplicación en Vitis



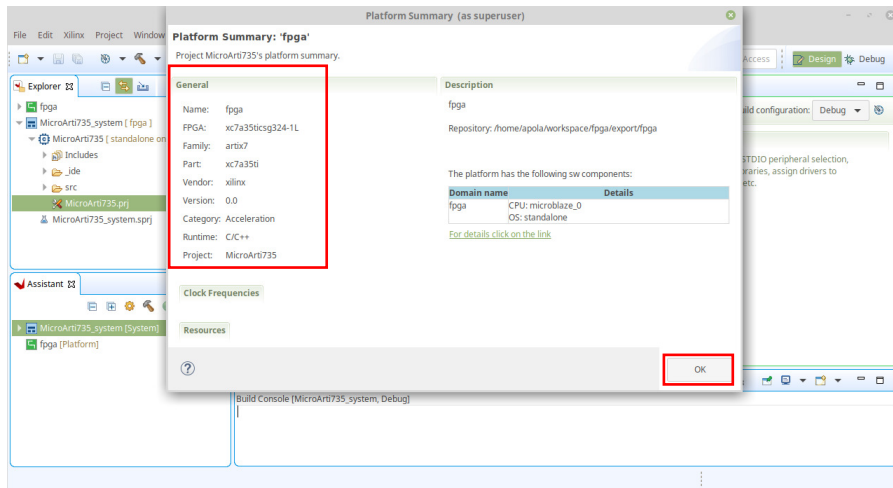
Asignar el proyecto Hello Word.

Crear la aplicación en Vitis



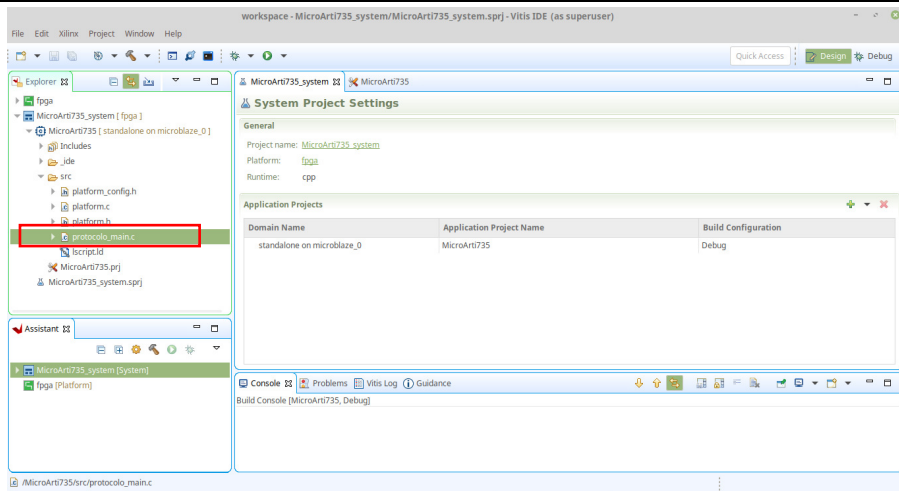
Se generan dos carpetas con el detalle del hardware y la aplicación en C seleccionada. Se puede verificar las características técnicas del diseño haciendo click en el nombre del hardware.

Crear la aplicación en Vitis



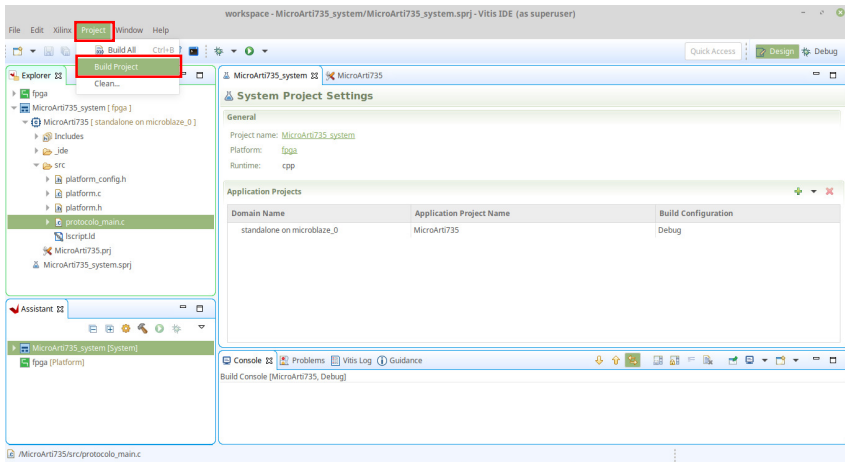
Detalles de la aplicación.

Crear la aplicación en Vitis



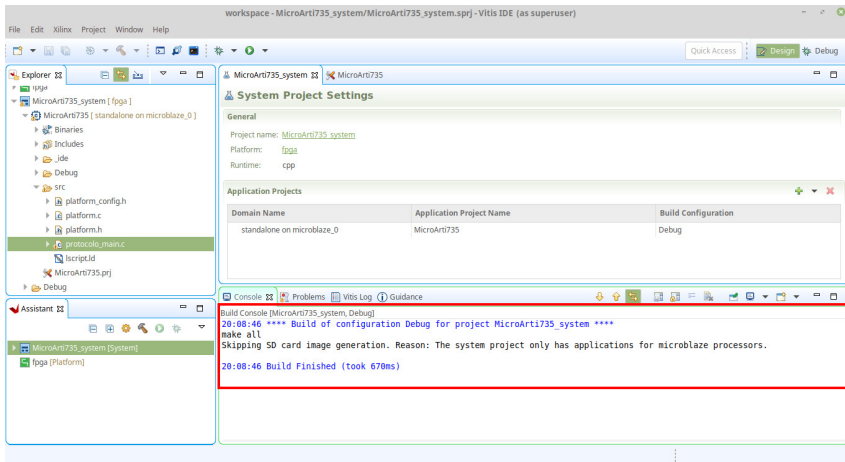
Reemplazar el archivo "Hello Word" con "protocolo_main" y "funciones.h" desde la dirección del proyecto. Ej. "/home/apola/workspace/MicroArti735/src/"

Crear la aplicación en Vitis



Compilar el proyecto (Build Project) o compilar todo Build All).

Crear la aplicación en Vitis

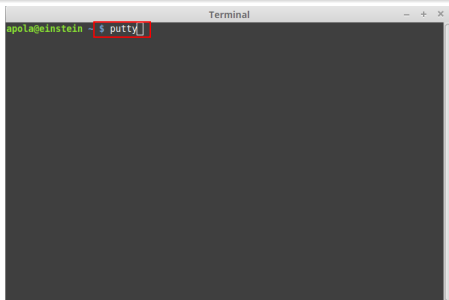


Verificar el estado de la compilación.

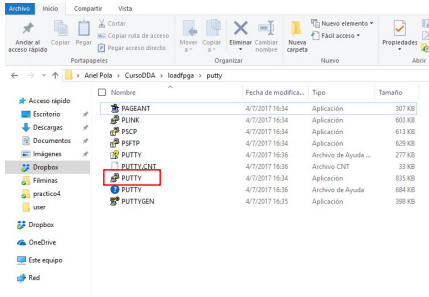
Túnel y Asignación de FPGA

Acceso Remoto a Servidor

- Instalar la herramienta Putty. En el caso de Windows se tendrán los ejecutables descargados desde <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>.
- Ejecutar
 - **Linux:** En un terminal ejecutar el programa Putty.
 - **Windows:** Hacer doble click sobre el icono de Putty.

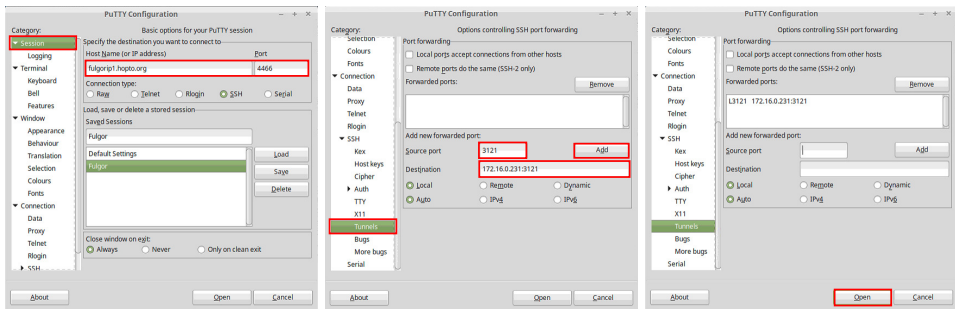


Linux



Windows

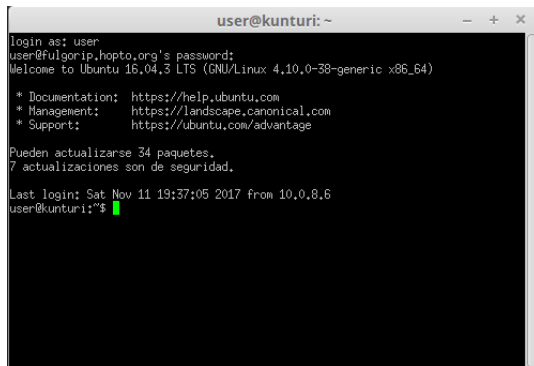
Tunel y Asignación de FPGA



- En la categoría **Session**, setear el host: *fulgorip.hopto.org* o *fulgorip1.hopto.org* y el puerto: *4466*.
- En la categoría **Connection** – **SSH** – **Tunnels** incluir el *forwarded* del puerto y agregarlo a la lista.
- Abrir el tunel.

Túnel y Asignación de FPGA

- Para el acceso se solicita usuario (**user**) y contraseña (**Cai1keaw**).
- Este acceso es para todos por igual.

A terminal window titled 'user@kunturi: ~' with standard window controls. It displays the login process for a user named 'user' on an Ubuntu 16.04.3 LTS system. The output includes the password prompt, a welcome message, system information, and links to documentation, management, and support. It also shows package update information and the last login time.

```
user@kunturi: ~
login as: user
user@fulgorip,hopto.org's password:
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-38-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Pueden actualizarse 34 paquetes.
7 actualizaciones son de seguridad.

Last login: Sat Nov 11 19:37:05 2017 from 10.0.8.6
user@kunturi:~$
```

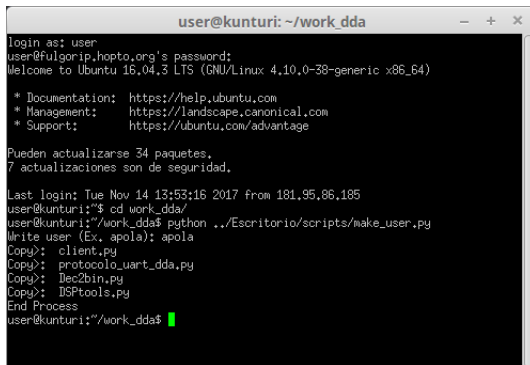
Tunel y Asignación de FPGA

- Para generar el entorno de trabajo debemos ejecutar la siguiente línea de comando dentro de la carpeta **work_dda** agregando el usuario personal con la inicial del nombre y el apellido completo (Ej. apola).

1 `cd work_dda`

2 `python ../Escritorio/scripts/make_user.py`

- El script copia unos archivos y arma el árbol de carpetas.



```
user@kunturi: ~/work_dda
login as: user
user@fulgorip.hopto.org's password:
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-38-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Pueden actualizarse 34 paquetes,
7 actualizaciones son de seguridad.

Last login: Tue Nov 14 13:53:16 2017 from 181.95.86.185
user@kunturi:~$ cd work_dda/
user@kunturi:~/work_dda$ python ../Escritorio/scripts/make_user.py
Write user (Ex. apola): apola
Copy>: client.py
Copy>: protocolo_uart_dda.py
Copy>: Dec2bin.py
Copy>: DSPtools.py
End Process
user@kunturi:~/work_dda$
```

Tunel y Asignación de FPGA

- Acceder a la carpeta `<user>/scripts/` y ejecutar **client.py** para solicitar un puerto USB y el ID de la placa FPGA disponible.

1 *cd apola/scripts*

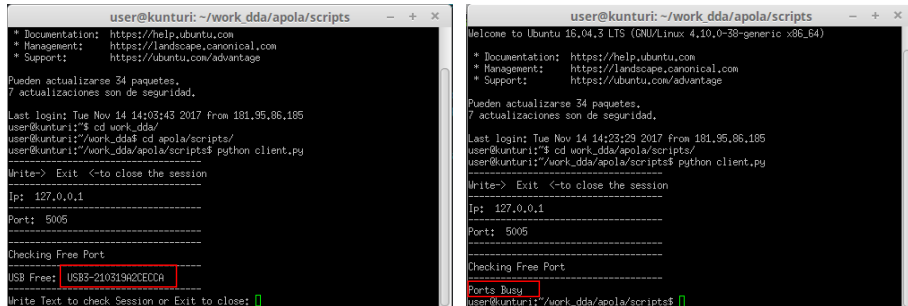
2 *python client.py*

- El script retorna el USB: **USB3** y el ID de la FPGA: **210319A2CECCA**.
- En caso de no haber disponible ninguna placa, el script retorna que los puertos estan ocupados.
- Esta ventana y el script no se tienen que cerrar hasta no terminar de usar la FPGA, ya que se libera el puerto y dará acceso a otro usuario.
- En las siguientes figuras se observa el caso de asignación correcta y puertos ocupados.

Nota: Copia de archivos o carpetas

- Usamos el comando **pscp** para copiar archivos entre sesiones.
- Remoto a local
`pscp -P 4466 user@fulgorip1.hopto.org://home/user/work_dda/<user>/<file> ./`
- Local a remoto
`pscp -P 4466 ./<file> user@fulgorip1.hopto.org://home/user/work_dda/<user>/`

Túnel y Asignación de FPGA



```
user@kunturi: ~/work_dda/apola/scripts
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage

Pueden actualizarse 34 paquetes,
7 actualizaciones son de seguridad.

Last login: Tue Nov 14 14:03:43 2017 from 181.95.86.185
user@kunturi:~$ cd work_dda/
user@kunturi:~/work_dda$ cd apola/scripts/
user@kunturi:~/work_dda/apola/scripts$ python client.py

Write-> Exit <-to close the session

Ip: 127.0.0.1
Port: 5005

Checking Free Port

USB Free: USB3-21031942CECDA
Write Text to check Session or Exit to close: [ ]
```

```
user@kunturi: ~/work_dda/apola/scripts
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-38-generic x86_64)

* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage

Pueden actualizarse 34 paquetes,
7 actualizaciones son de seguridad.

Last login: Tue Nov 14 14:23:29 2017 from 181.95.86.185
user@kunturi:~$ cd work_dda/apola/scripts/
user@kunturi:~/work_dda/apola/scripts$ python client.py

Write-> Exit <-to close the session

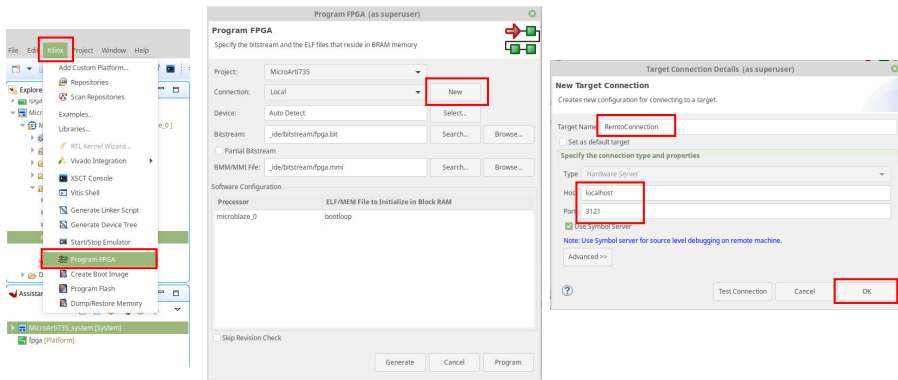
Ip: 127.0.0.1
Port: 5005

Checking Free Port

Ports Busy
user@kunturi:~/work_dda/apola/scripts$ [ ]
```

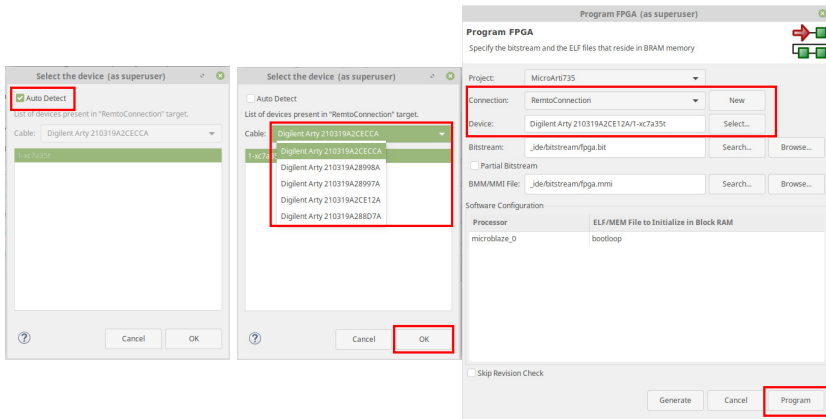
Las figuras muestran la asignación correcta de puerto (izq.) y los puertos ocupados (der.).

Programar la FPGA



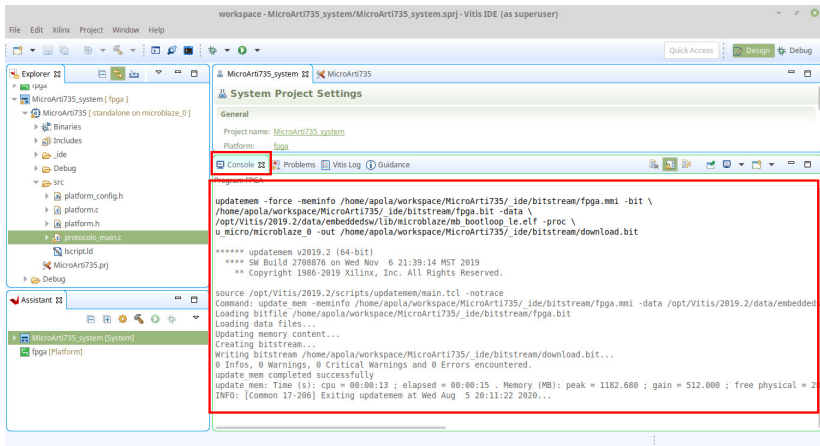
Programar la FPGA. Crear un nuevo Server, dar un nombre y colocar la dirección IP del server remoto. Para nuestro caso, el cual realiza un forward del puerto 3121 se debe colocar LOCALHOST como dirección de host.

Programar la FPGA



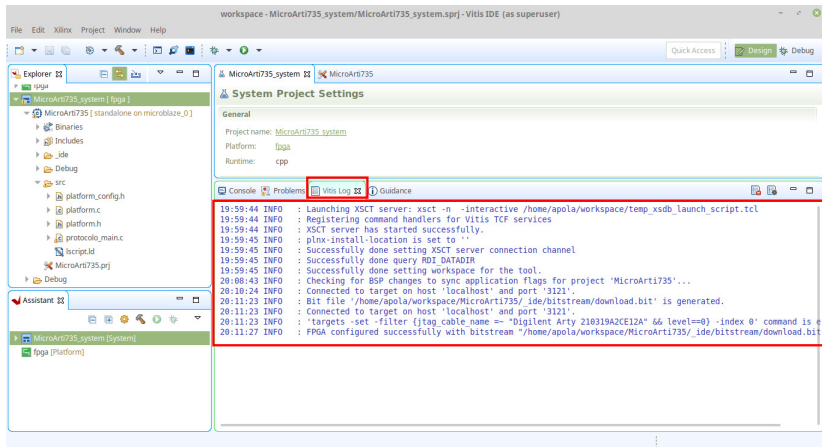
Desactivar AutoDetect, seleccionar el ID de la FPGA y verificar la información.

Programar la FPGA



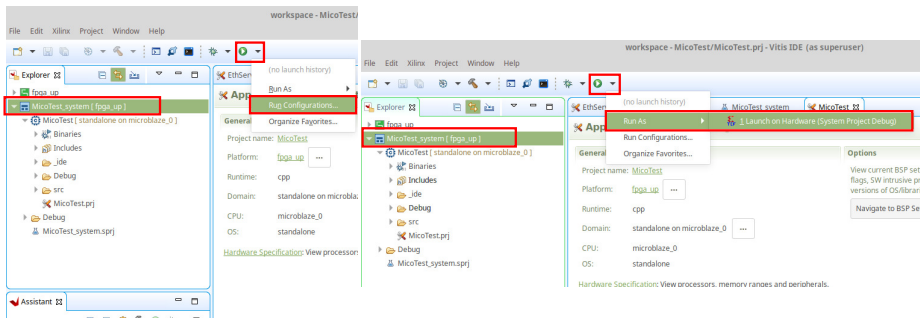
En la “consola” se observará la generación de un nuevo binario.

Programar la FPGA



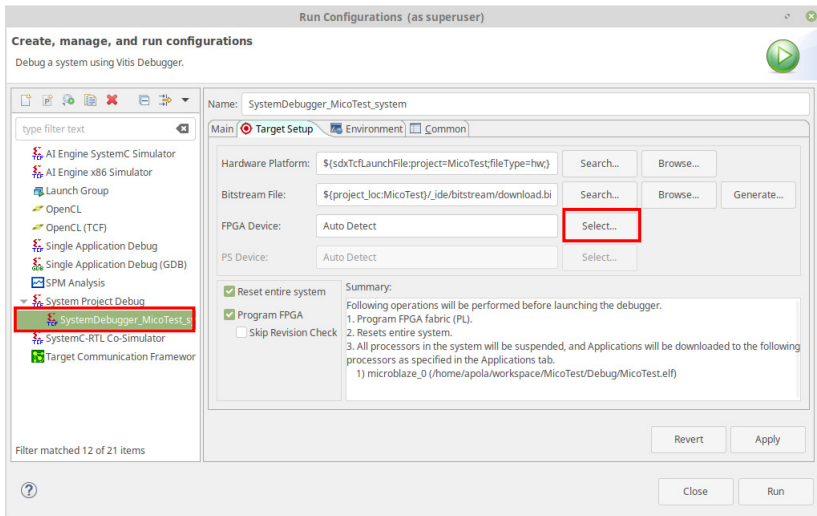
En "Vitis Log" se detallará la programación.

Ejecutar la Aplicación



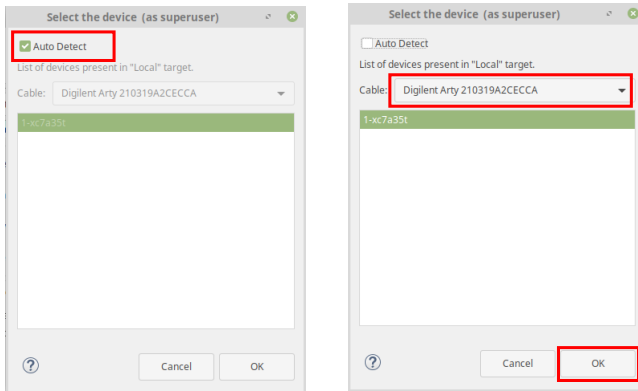
Antes de ejecutar la aplicación es necesario setear el dispositivo que se quiere correr. Para ello necesitamos acceder a Run Configuration. *Nota: En caso que no aparezcan las opciones que se detallan en la figura de la siguiente pagina, debemos ejecutar al menos una vez la aplicación. En este caso debemos seguir los pasos de la figura de la derecha.*

Ejecutar la Aplicación



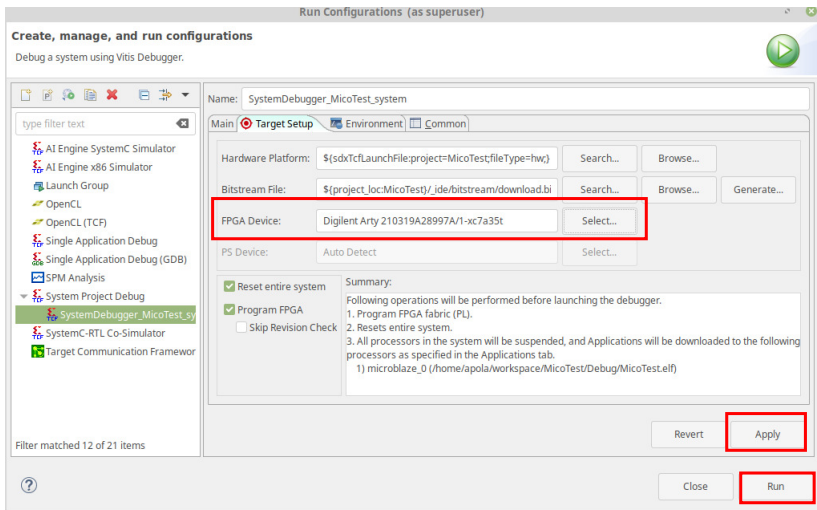
En la opción SystemDebugger, cambiar el dispositivo (FPGA Device).

Ejecutar la Aplicación



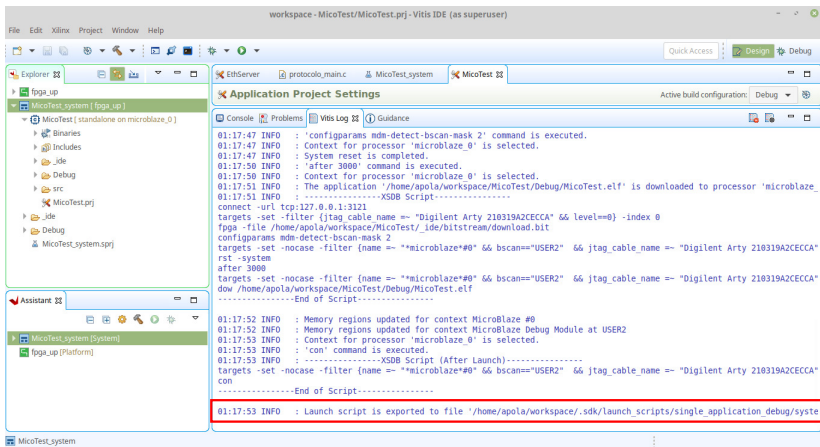
Seleccionar una FPGA.

Ejecutar la Aplicación



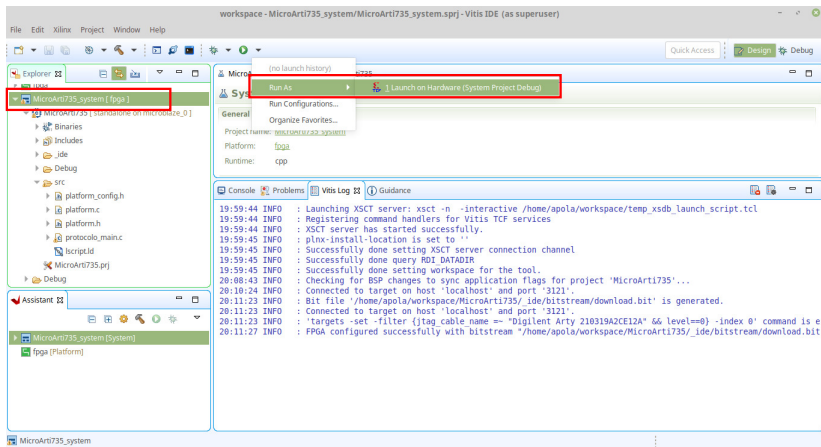
Verificar el ID, aplicar los cambios y ejecutar.

Ejecutar la Aplicación



El mensaje Launch verifica que se esta corriendo la aplicación en el microProcesador.

Ejecutar la Aplicación



En caso de necesitar ejecutar varias veces la aplicación y una vez que se han ejecutado los pasos anteriores, basta con lanzar nuevamente la aplicación.

Proyecto FSM y BRAM

```
user@wanpu: ~/work_dda/apolaDDA2024/scripts
File Edit View Search Terminal Help
apola@newton:~$ ssh -L 3121:172.16.0.231:3121 user@fulgorip.hopto.org -p 4466
user@fulgorip.hopto.org's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.8.0-49-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Expanded Security Maintenance for Applications is not enabled.

128 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

7 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Dec 13 21:34:39 2024 from 181.4.21.115
user@wanpu:~$ cd work_dda/apolaDDA2024/scripts/
user@wanpu:~/work_dda/apolaDDA2024/scripts$ cp ../../Desktop/unit06/protocolo_ct_conf.py ./
user@wanpu:~/work_dda/apolaDDA2024/scripts$
```

Copiar en la carpeta de trabajo los scripts **protocolo_ct_conf.py**, **Dec2bin.py**, **DSPtools.py** desde el escritorio del servidor. Además, crear la carpeta logs en el mismo lugar donde se ejecuta el script (**mkdir ./logs**).

Proyecto FSM y BRAM

```
user@kunturi: ~/work_dda/apola/scripts
user@kunturi:~/work_dda/apola/scripts$ python protocolo_ct_conf.py 5 (1)
Log File Name: SavaData (2)
Log Ram Width (8/16/32): 32 (3)
Log NB,NBF (Ex: 8,7): 32,0 (4)
RESET ON
MODULES OFF
LOG RUN OFF
RESET OFF
ENB MODULES
Value: 2 (5)
LOG
LOG RUN OFF
LOG RUN ON
LOG READ
4096
End Script
user@kunturi:~/work_dda/apola/scripts$
```

Ejecutar el script que se comunica con el micro y lee los datos guardados en la memoria. (1) Puerto USB. (2) Nombre del archivo de logueo. (3) Número de bits que se agruparán por columna en el archivo de logueo. (4) Representación en punto fijo de la resolución de loa datos loguados. (5) Comando que determina el paso del sumador. Los resultados se guardarán en **<path>/logs/<fileName.out>**